

■ Modular Seasonal Crop Advisor

MSCA — Bring Your Own Project (BYOP) Capstone Report

Student Name	Rudra Patel
Project Title	Modular Seasonal Crop Advisor (MSCA)
Language	Python 3.x
Interface	Command Line Interface (CLI)
Entry File	seasonal_crop_advisor.py
GitHub	github.com/rudrapatel7019/Seasonal_crop_advisor
Submission	March 2026

1. Problem Statement

Many novice and intermediate gardeners face difficulty tracking the correct planting windows for different crops throughout the year. Without reliable guidance, they plant the wrong crops in the wrong season — leading to reduced yields, wasted seeds, and lost effort.

There is a clear need for a quick, accessible, and error-free tool that provides instant, season-specific crop recommendations. The tool should require no installation beyond Python itself, no internet connection, and no prior agricultural knowledge to operate.

The Modular Seasonal Crop Advisor (MSCA) addresses this by accepting a season name from the user and immediately returning a formatted, numbered list of crops well-suited for that season — with robust error handling for any invalid input.

2. Scope of the Project

This project is a proof-of-concept CLI application built in Python, providing generalized crop suggestions based on the four major meteorological seasons. The primary focus is clean modular code structure and robust input validation.

Season	Crops in Dataset
Spring	Lettuce (Leafy Greens), Peas, Radishes, Spinach, Broccoli (starting indoors), Potatoes
Summer	Tomatoes, Peppers (Bell and Chili), Zucchini and Squash, Corn, Cucumbers, Beans (Pole and Bush), Eggplant
Autumn	Kale, Carrots, Garlic (for overwintering), Pumpkins and Winter Squash, Cabbage, Beets
Winter	Brussels Sprouts, Leeks, Hardy Kale and Spinach (under cover), Overwintered Garlic, Cover Crops (e.g., Rye)

Out of Scope (Future Enhancements): Regional/geographic data, soil type analysis, disease prediction, GUI or web interface, weather API integration.

3. Target Users

- Beginner and intermediate home gardeners who need simple, reliable seasonal planting guidance.
- Students seeking a quick, accurate crop reference tool.
- Python learners who want a clean, functional single-file project as a reference example.

4. Architecture & Modular Design

The application lives in a single file (`seasonal_crop_advisor.py`) but is internally structured into three clearly separated functional modules — demonstrating the separation of concerns principle without requiring multiple files.

Module	Function	Responsibility
Data Module	<code>get_default_crop_data()</code>	Returns the crop dictionary — the entire dataset lives here, isolated from all logic
Utility Module	<code>suggest_crops(season, crop_data)</code>	Validates season input, performs dictionary lookup, prints formatted numbered crop list or error message
Application Module	<code>main()</code>	Loads crop data, displays welcome message, runs the while True input loop, handles exit and empty input edge cases

4.1 Data Module — `get_default_crop_data()`

Holds the crop dataset as a Python dictionary with lowercase season names as keys and lists of crop strings as values. Dictionary storage enables $O(1)$ constant-time lookup. The data is completely isolated from processing logic — updating the knowledge base requires editing only this one function.

4.2 Utility Module — `suggest_crops()`

Accepts a season string and the crop data dictionary. Converts input to lowercase using `.lower()` for case-insensitive matching, checks if the key exists, and either prints a numbered crop list using `enumerate(suggestions, 1)` or an informative error message. Never crashes on invalid input.

4.3 Application Module — `main()`

Entry point that loads crop data, displays a welcome banner, and enters a while True loop. Handles three cases: 'exit' typed (clean termination), blank input (re-prompts), or a season name (delegates to `suggest_crops()`). Wrapped in try/except with `sys.exit(1)` for unexpected errors.

5. Technology Stack

Component	Detail
Language	Python 3.x
Core Data Structure	Python Dictionary — $O(1)$ constant-time key-value lookup
Input Handling	<code>input()</code> with <code>.strip()</code> and <code>.lower()</code> for normalization
Output Formatting	<code>enumerate(suggestions, 1)</code> for clean numbered lists

Error Handling	try/except block in main() with sys.exit(1)
External Dependencies	None — Python standard library only (sys module)
Version Control	Git and GitHub

6. Key Decisions Made

Dictionary over if/elif chain

The season lookup could have been implemented as a chain of if/elif statements. A dictionary was chosen instead for O(1) lookup time, cleaner code, and because the data layer can be expanded without modifying the logic layer.

.lower() for case normalization

Evaluated .upper(), .lower(), and .title(). Since dictionary keys are stored in lowercase, .lower() was the most direct match — zero conversion overhead, maximum clarity.

Single file, modular functions

A multi-file structure (data.py, utils.py, main.py) was considered but a single-file approach was chosen for simplicity of distribution and execution. Modularity is achieved through clearly named functions rather than file separation.

sys.exit(1) for runtime errors

Rather than letting Python print a raw traceback on unexpected errors, the main() function wraps all logic in try/except and exits with code 1 — following Unix conventions and providing a cleaner user experience.

No external dependencies

Libraries like click or rich were considered for a richer CLI experience. Zero-dependency was chosen deliberately so the tool runs on any Python 3 installation with no setup steps — prioritising accessibility over aesthetics.

7. Testing Approach

The application was validated through structured manual testing covering positive, edge, and negative cases:

Test Case	Input	Expected Result	Status
Positive Test	Spring	Prints numbered list of 6 spring crops	✓ Pass
Edge Case — case sensitivity	aUtUmN	Matches 'autumn', prints autumn crop list	✓ Pass
Negative Test — invalid input	monsoon	Error: 'monsoon is not a recognized season'	✓ Pass
Empty input guard	(blank)	Re-prompts user without crashing	✓ Pass
Exit flow control	exit	Goodbye message, clean termination	✓ Pass

8. Course Concepts Applied

Concept	Application in This Project
Python Dictionaries	Core data structure for O(1) season-to-crop lookups — chosen deliberately over if/elif or list iteration.
Functions & Modularity	Three functions each with a single, clear responsibility: data, processing, interface.
Loops	while True loop in main() for continuous user interaction until explicit exit.
Conditionals	Input validation in suggest_crops() and edge case handling in main().
String Methods	.lower() and .strip() for input normalization; .capitalize() for display formatting.
enumerate()	Used in suggest_crops() to produce clean numbered output lists starting at 1.
Exception Handling	try/except in main() with sys.exit(1) for runtime error management.
sys module	Imported for robust program exit control with standard Unix exit codes.
Docstrings	All three functions include proper Python docstrings with Args and Returns documentation.
Version Control	Iterative development tracked with Git commits and hosted on GitHub.

9. Challenges Faced

- **Input normalization strategy:** Choosing between .lower(), .upper(), and .title() required evaluating each against the dictionary key format. .lower() was chosen as the most direct match to lowercase keys — fewest conversion steps, maximum clarity.
- **Empty input edge case:** Early versions passed blank strings to suggest_crops(), producing a confusing 'empty string is not a recognized season' error. An explicit empty-string check in main() was added to handle this gracefully.
- **Single-file vs multi-file structure:** Balancing modularity (which typically implies separate files) against simplicity of a single executable. The solution was three clearly named functions acting as logical modules within one file.
- **Scope discipline:** The temptation to add features — weather API, regional data, a Streamlit UI — had to be actively resisted to keep the project focused and the code clean.

10. What I Learned

- Modular thinking in a single file: clear function names with single responsibilities are sufficient for a project of this scope — multiple files are not always necessary.

- Data structure choice has real consequences: a dictionary lookup is simpler, faster, and more readable than nested conditionals, even for a small dataset.
- Error handling is part of the user experience: a clear, non-crashing error message is more valuable than a technically correct traceback.
- Docstrings are documentation: writing Args and Returns for each function forced clarity about what each function actually does and expects.
- Simplicity is a deliberate choice: a tool that does one thing well and runs anywhere is better engineering than an over-featured system.

11. Future Scope

- Move the crop dictionary from `get_default_crop_data()` into an external `crops.json` or `crops.csv` file for easier updates.
- Add an optional region/country parameter for location-specific recommendations.
- Include growing period, water requirements, and soil type notes alongside each crop name.
- Replace manual test cases with a pytest test suite for automated continuous validation.
- Wrap the core `suggest_crops()` logic in a Streamlit web app for browser-based access.

12. Conclusion

The Modular Seasonal Crop Advisor is a focused, well-designed solution to a genuine everyday problem. Every design decision — using a dictionary for $O(1)$ lookup, separating data from logic from interface, handling all input edge cases, and wrapping the program in proper error handling — was made deliberately and can be clearly justified.

The project demonstrates that good software engineering is not about complexity. A clean, single-file Python script that is instantly usable by anyone with Python installed, handles errors gracefully, and is straightforward to extend is exactly what this capstone asked for: a purposeful, well-executed solution to a real problem.

Step-by-Step Submission Checklist

1	Make Repo Public Go to github.com/rudrapatel7019/Seasonal_crop_advisor → Settings → scroll to Danger Zone → Change visibility → Make public
2	Add README to Repo Download README.md from this submission → drag and drop into your GitHub repo root → commit with message: 'Add README for BYOP submission'
3	Check Commit History Make sure your repo has multiple commits (not just one upload). Add at least 2-3 commits showing development stages if needed.
4	Verify Code Runs Run: <code>python seasonal_crop_advisor.py</code> — test all 5 cases: Spring, aUtUmN, monsoon, blank enter, exit
5	Submit on VITyarthi Upload: (a) GitHub repo link, (b) this PDF Project Report, (c) confirm README is live in the repo

GitHub: github.com/rudrapatel7019/Seasonal_crop_advisor